VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ
FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ
DEPARTMENT OF TELECOMMUNICATIONS

ZABEZPEČENÍ DAT POMOCÍ ALGORITMŮ POSTKVANTOVÉ KRYPTOGRAFIE

SEMESTRÁLNÍ PROJEKT
SEMESTRAL PROJECT

AUTOŘI PROJEKTU
AUTHORS

Martin Szüč, Zakhar Vasiukov, Pavlo Balan, Branislav Kadlec

BRNO 2026

Obsah

Úvod **3**

1 Teorie **4**

1.1 Bezpečnostní cíle 4
1.2 Ustavení klíče a KDF 4
1.3 Digitální podpisy 4
1.4 Hybridní konfigurace 5
1.5 AEAD šifrování 5

2 Praktická část **6**

2.1 Technologický zásobník a nasazení 6
2.2 Uživatelské rozhraní 6
2.3 Handshake protokol 7
2.4 Zabezpečený přenos dat 8
2.5 Logování a auditní stopa 9
2.6 Limity implementace a budoucí rozvoj 9

Závěr **11**

Literatura **12**

Úvod

Rozvoj postkvanové kryptografie je přímou reakcí na očekávané bezpečnostní dopady existence kvantových počítačů na tradiční asymetrické schémata. Zejména kryptografie založená na problému diskrétního logaritmu je v kvantovém modelu ohrožena algoritmem, který vede k polynomiálnímu řešení původně exponenciálně obtížných úloh. V praxi to znamená, že současné protokoly zabezpečené komunikace musí být navrhovány s ohledem na schopnost systému pružně měnit použité kryptografické primitivum bez zásahu do architektury.

Tento semestrální projekt zpracovává demonstrační webovou aplikaci, která simuluje P2P komunikaci mezi dvěma účastníky **Alice** a **Bob**. Cílem je implementovat vrstvený návrh, kde:

- handshake poskytuje ustavení sdíleného tajemství a autentizaci protistrany,
- přenos dat je realizován symetrickým šifrováním,
- aplikační rozhraní umožní volbu algoritmů,
- systém zaznamenává bezpečnostní události pro účely auditu a diagnostiky.

Implementace je realizována v jazyce Python s využitím frameworku Flask. Kryptografická vrstva je navržena jako plně modulární a nabízí výběr algoritmů ve třech základních kategoriích: mechanismy pro ustavení klíčů (`ML-KEM-768` nebo `ECDH P-256`), digitální podpisy (`ML-DSA-65` nebo `ECDSA P-256`) a symetrické šifry (`AES-256-GCM` nebo `ChaCha20-Poly1305`). Architektura systému umožňuje uživateli pro každou relaci zvolit libovolnou kombinaci těchto prvků, což dovoluje vytvářet čistě klasické, postkvantové i hybridní konfigurace.

1 Teorie

Tato kapitola shrnuje základní pojmy a kryptografické stavební bloky použité v projektu. Důraz je kladen na propojení klasických a postkvantových primitiv do uceleného a modulárního komunikačního schématu.

1.1 Bezpečnostní cíle

V rámci bezpečné komunikace na aplikační vrstvě sledujeme zejména následující vlastnosti:

- **Důvěrnost** — obsah zprávy nesmí být bez znalosti klíče relace čitelný.
- **Integrita** — jakákoliv modifikace přenášených dat musí být detekovatelná.
- **Autenticita** — ověření identity protistrany a původu dat.

1.2 Ustavení klíče a KDF

Proces ustavení sdíleného tajemství v projektu využívá dva odlišné přístupy v závislosti na zvolené konfiguraci:

- **ECDH** — klasický mechanismus založený na eliptických křivkách.
- **ML-KEM** — postkvantový mechanismus založený na mřížkách.

Získané syrové tajemství je následně zpracováno funkcí HMAC-based Key Derivation Function s využitím SHA-256. HKDF zajišťuje extrakci kryptograficky silného materiálu a jeho následné rozprostření do klíčů pro symetrické šifrován Získané syrové tajemství je následně zpracováno funkcí **HKDF** s využitím SHA-256. HKDF zajišťuje extrakci kryptograficky silného materiálu a jeho následné rozprostření do klíčů pro symetrické šifrování.

1.3 Digitální podpisy

Digitální podpisy v aplikaci slouží k autentizaci handshake a zajištění neodpíratelnosti identity.

- **ECDSA P-256** — standardní algoritmus využívající eliptické křivky, poskytující vysokou efektivitu při malých délkách klíčů.
- **ML-DSA** — postkvantový podpisový standard založený na mřížkách.

1.4 Hybridní konfigurace

Vzhledem k přechodnému období mezi klasickou a postkvantovou kryptografií aplikace umožňuje tzv. **hybridní režim**. V tomto režimu dochází ke kombinaci obou světů (např. ECDH + ML-KEM). Výsledné klíče jsou derivovány z obou vstupů současně, což zaručuje, že komunikace zůstane bezpečná, dokud je bezpečný alespoň jeden z použitých algoritmů.

1.5 AEAD šifrování

Pro samotný přenos dat jsou využity režimy Authenticated Encryption with Associated Data (AEAD), které garantují důvěrnost i integritu v rámci jedné operace:

- **AES-256-GCM** — průmyslový standard s hardwarovou podporou na většině moderních CPU.
- **ChaCha20-Poly1305** — moderní alternativa, která vykazuje vysoký výkon v čistě softwarových implementacích (např. mobilní zařízení).

Oba režimy pracují s unikátním inicializačním vektorem (nonce), aby se předešlo útokům založeným na opakování klíče.

2 Praktická část

Tato kapitola detailně popisuje architekturu, implementační detaily a bezpečnostní analýzu demonstrační aplikace. Systém simuluje zabezpečenou P2P komunikaci v hybridním prostředí, které kombinuje klasickou kryptografii s postkvantovými standardy (PQC). Cílem praktické části je demonstrovat, že přechod na kvantově odolné algoritmy lze realizovat modulárně díky abstrakci kryptografických vrstev.

2.1 Technologický zásobník a nasazení

Aplikace je vyvinuta v jazyce **Python 3.12**. Volba tohoto ekosystému umožnila efektivní orchestraci nativních knihoven a poskytla flexibilitu pro integraci různých implementací PQC algoritmů.

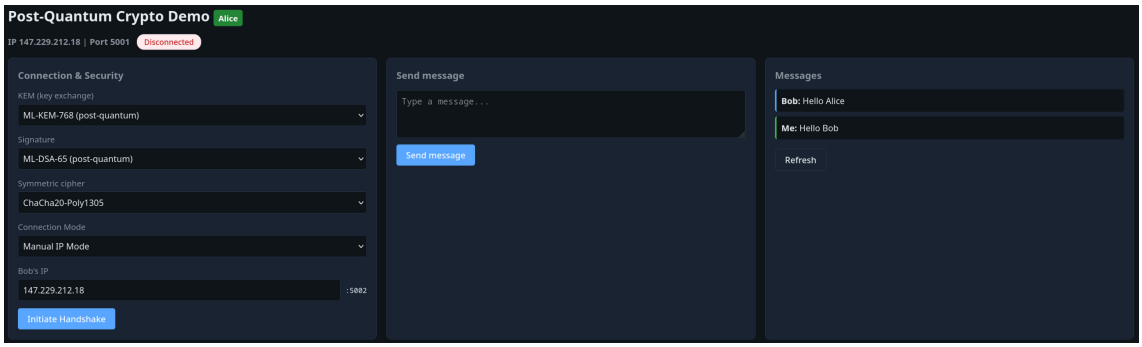
- **Flask a síťová topologie**: Každý účastník běží jako samostatná instance. V lokálním prostředí je **Alice** mapována na port **5001** a **Bob** na port **5002**. Tato izolace procesů věrně simuluje distribuované prostředí, kde uzly sdílejí pouze veřejnou síť a komunikují skrze REST API endpointy.
- **Kryptografické jádro**:
  - Pro klasické algoritmy je využita knihovna `cryptography`.
  - Postkvantová vrstva využívá knihovnu `mlkem` pro optimalizovanou implementaci mechanismu ustavení klíče a `dilithium-py` pro digitální podpis.
- **Kontejnerizace a Docker**: Vzhledem k tomu, že `liboqs` vyžaduje kompilaci nativních knihoven v jazyce C (k čemuž jsou zapotřebí nástroje `cmake` a `gcc`), aplikace podporuje kontejnerizaci pomocí `Dockeru`. Tím je zaručeno, že aplikaci lze spustit na jakékoli platformě. Aplikace však podporuje i nativní spuštění v systému, za předpokladu, že je vytvořeno prostředí pro Python kód.

2.2 Uživatelské rozhraní

Pro demonstraci a interaktivní testování navrženého kryptografického systému byla realizována webová aplikace, která slouží jako hlavní řídicí panel (viz obr. 2.1).

V rámci inicializace relace uživatel definuje volí kombinaci kryptografických primitiv pro KEM, digitální podpis a symetrickou šifru. Po kliknutí na tlačítko Initiate Handshake aplikace provede sekvenci ustavení klíčů, jejímž výsledkem je vytvoření zabezpečeného šifrovaného tunelu. Jakmile je relace aktivní, ovládací panel se dynamicky transformuje do stavu monitorování.

Funkce ukončení relace iniciuje tlačítko Terminate Session. Funkce odesílá podepsaný požadavek protistraně. Tím je zajištěno, že po ukončení komunikace jsou

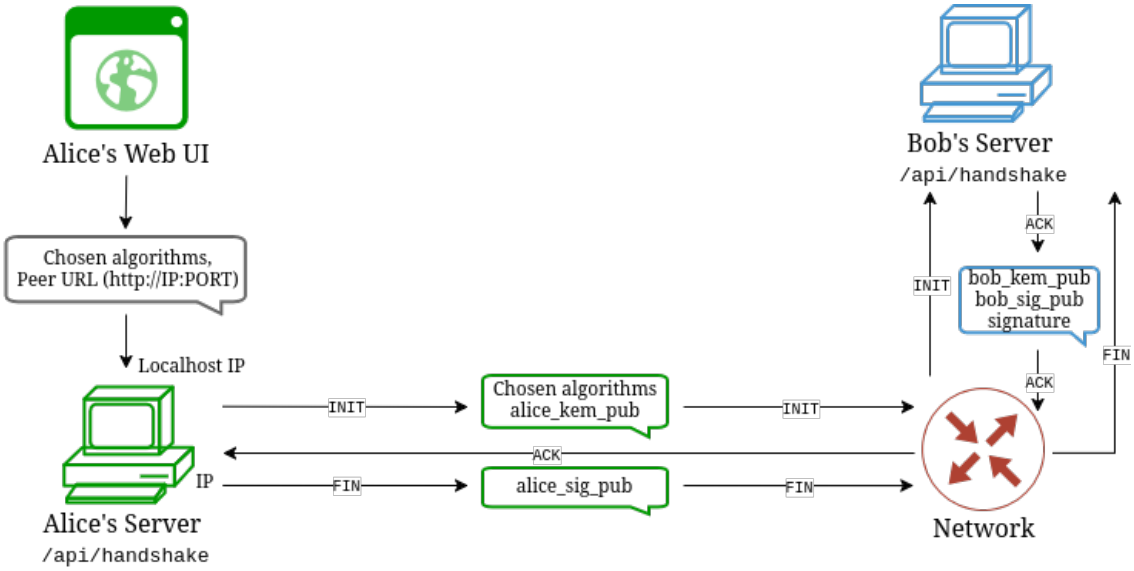


Obr. 2.1: Uživatelské rozhraní aplikace Alice.

relační klíče a citlivá data z operační paměti obou uzlů bezpečně odstraněny, čímž se předchází jejich případnému zneužití při kompromitaci zařízení v budoucnu.

2.3 Handshake protokol

Navržený handshake protokol (viz obr. 2.2) zajišťuje bezpečné ustavení sdíleného tajemství a vzájemnou autentizaci entit v prostředí s hrozbou kvantových počítačů. Proces využívá kombinaci mechanismu zapouzdření klíče (KEM) a digitálních podpisů.



Obr. 2.2: Schéma handshake procesu.

Proces je rozdělen do tří hlavních fází:

1. **Fáze INIT (Client Hello):** Proces začíná v uživatelském rozhraní, kde jsou definovány parametry spojení. Klientský uzel generuje pár klíčů pro zvolený KEM. Následně odesílá na endpoint `/api/handshake` serveru zprávu obsahující zvolená kryptografická primitiva a svůj veřejný klíč `alice_kem_pub`.
2. **Fáze ACK (Server Hello):** Bob po přijetí požadavku provede zapouzdření (encapsulation) proti veřejnému klíči Alice, čímž vznikne šifrový text (`bob_kem_pub` / *ciphertext*) a sdílené tajemství. Bob dále generuje svůj podpisový pár klíčů a vytvoří digitální podpis nad celým dosavadním transkriptem komunikace. Tímto krokem je zajištěna integrita handshake a ochrana proti útoku *Man-in-the-Middle* (MitM). Alice obdrží veřejné klíče Boba a podpis transkriptu.
3. **Fáze FIN (Client Finish):** Alice verifikuje Bobův podpis. V případě úspěšné validace provede odpuštění (decapsulation) a získá identické sdílené tajemství. Následně odesílá Bobovi svůj veřejný podpisový klíč `alice_sig_pub`, čímž je ustavena důvěra pro následnou obousměrnou komunikaci.

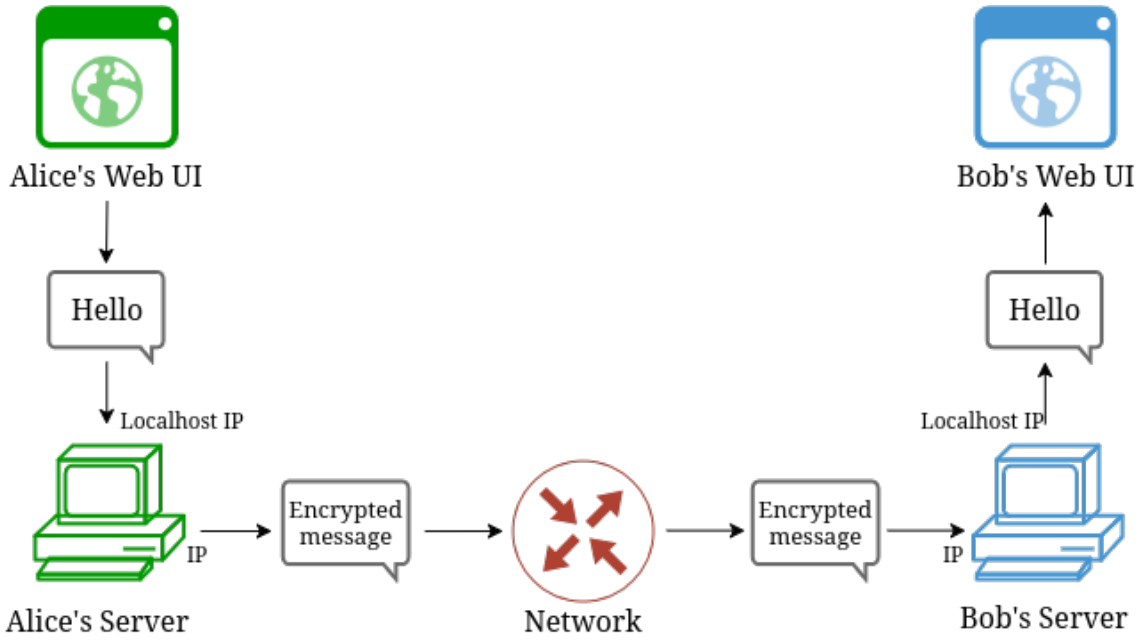
2.4 Zabezpečený přenos dat

Po úspěšném ustavení sdíleného tajemství v rámci handshake protokolu přechází aplikace do fáze aktivního přenosu dat. Jak je znázorněno na schématu (viz obr. 2.3), proces transformace prostého textu na zabezpečený payload probíhá na straně serveru odesílatele před jeho odesláním do nezabezpečené sítě.

Implementace využívá paradigma **Encrypt-then-Sign**, které zajišťuje komplexní ochranu zprávy:

- **Symetrické šifrování (Důvěrnost):** Zpráva je nejprve zašifrována pomocí zvoleného algoritmu (např. AES-GCM nebo ChaCha20-Poly1305) s využitím 256bitového relačního klíče odvozeného během handshake.
- **Digitální podpis (Integrita a Autenticita):** Nad zašifrovaným payloadem je následně vygenerován podpis.

Obrázek 2.3 vizualizuje kompletní cestu zprávy: od vstupu v uživatelském rozhraní (*Web UI*), přes šifrování a podepisování na serveru odesílatele, až po doručení protistraně, která provádí inverzní operace (verifikaci podpisu a následné dešifrování) před zobrazením obsahu uživateli.



Obr. 2.3: Vizualizace bezpečného přenosu dat.

2.5 Logování a auditní stopa

Každý uzel udržuje `security.log`, který slouží k diagnostice a auditu. Logy jsou strukturovány tak, aby umožnily rekonstrukci bezpečnostních událostí bez úniku tajných klíčů:

- Záznam o startu instance na portu `5001/5002`.
- Detaily o každém handshaku (použitý KEM, Signature algoritmus).
- Hlášení o chybách verifikace (např. neplatný podpis nebo chybný AEAD tag), což indikuje pokus o útok.

2.6 Limity implementace a budoucí rozvoj

Aktuální aplikace slouží jako Proof-of-Concept postkvantové komunikace. V rámci vývoje a testování byly identifikovány následující oblasti, které představují technologická omezení a definují směr pro budoucí rozvoj systému:

- **Absence validace certifikačních autorit:** Systém v současné verzi neprovádí ověřování veřejných klíčů proti důvěryhodné třetí straně. Vzhledem k absenci infrastruktury certifikačních autorit aplikace v podstatě důvěřuje jakémukoliv peeru, který úspěšně dokončí handshake. Produkční nasazení by vyžadovalo implementaci validačních řetězců, které by zajistily, že prezentovaný veřejný klíč je skutečně svázán s danou identitou.

- **Parametrizace sítě a algoritmů:** Aktuální verze využívá statickou síťovou konfiguraci s pevně definovanými porty (`5001` a `5002`). Budoucí vývoj by měl umožnit plnou uživatelskou volbu IP adres a portů přímo v UI. Dále se nabízí rozšíření o jemnější nastavení kryptografických parametrů, jako je volba konkrétních variant bezpečnostních úrovní (např. ML-KEM-512 vs. 1024) nebo délky nonce u symetrických šifer.
- **Ochrana proti útoku opakováním:** Protokol v současné verzi neimplementuje kontrolu sekvenčních čísel paketů. Útočník by tak mohl zachycený a validně podepsaný payload odeslat znovu. Zavedení monotónních čítačů v rámci zašifrovaného kontextu by zajistilo, že každá zpráva bude v rámci jedné relace akceptována právě jednou.

Závěr

V rámci tohoto projektu byla navržena a implementována klientská aplikace pro zabezpečenou P2P komunikaci. Výsledné řešení úspěšně propojuje modulární kryptografickou vrstvu s interaktivním webovým rozhraním, což umožňuje uživateli transparentně konfigurovat a sledovat celý životní cyklus zabezpečené relace.

Klíčovým přínosem práce je plná implementace postkvantových algoritmů dle nejnovějších standardů — konkrétně mechanismu pro zapouzdření klíče `ML-KEM-768` a digitálního podpisu `ML-DSA-65`. Systém byl navržen tak, aby umožňoval hybridní provoz a snadnou záměnu za klasická primitiva založená na eliptických křivkách. Ustavení sdíleného tajemství probíhá skrze robustní třífázový handshake, po němž následuje derivace relačních klíčů pomocí funkce `HKDF-SHA256`.

Samotná přenosová vrstva realizuje paradigma *Encrypt-then-Sign* s využitím moderních AEAD režimů (`AES-256-GCM` a `ChaCha20-Poly1305`). Nad rámec běžné komunikace byl do systému integrován mechanismus autentizovaného ukončení relace. Ten využívá stávající kryptografický kontext k odeslání podepsaného signálu k ukončení spojení, čímž je garantováno synchronní a bezpečné odstranění relačních klíčů z paměti obou uzlů.

Projekt prokázal, že navržená architektura zajišťuje důvěrnost, integritu a autenticitu dat i v prostředí ohroženém budoucím rozvojem kvantových výpočtů.

Literatura

[1] NIST. *Post-Quantum Cryptography Standardization*. Dostupné z: https://csrc.nist.gov/projects/post-quantum-cryptography. [cit. 2026-04-26].

[2] KRAWCZYK, Hugo; ERENKRANTZ, Pasi. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010. Dostupné z: https://www.rfc-editor.org/rfc/rfc5869. [cit. 2026-04-26].

[3] MCGREW, David; VIEGA, John. *An Interface and Algorithms for Authenticated Encryption*. RFC 5116, 2008. Dostupné z: https://www.rfc-editor.org/rfc/rfc5116. [cit. 2026-04-26].

[4] NIR, Yoav; LANGLEY, Adam. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, 2018. Dostupné z: https://www.rfc-editor.org/rfc/rfc8439. [cit. 2026-04-26].

[5] NIST. *FIPS 197: Advanced Encryption Standard (AES)*. 2001. Dostupné z: https://csrc.nist.gov/publications/detail/fips/197/final. [cit. 2026-04-26].

[6] Python Cryptographic Authority. *cryptography — Python library for cryptographic primitives*. Dostupné z: https://cryptography.io/. [cit. 2026-04-26].

[7] PALLETTS projects and contributors. *Flask Documentation*. Dostupné z: https://flask.palletsprojects.com/. [cit. 2026-04-26].